Análise Experimental da Inferência SepConv para Interpolação de Quadros de Vídeo com Ênfase em Eficiência Computacional e Uso de VRAM

Pedro Henrique Gomes

2026

SENAC Santo Amaro
Bacharelado em Ciência da Computação

Agenda

- Introdução;
- Hipótese Científica;
- Objetivos;
- Pesquisa Bibliográfica;
- Trabalhos Relacionados;
- Fundamentação Teórica;
- Desenvolvimento;
- Metodologia Experimental;
- Cronograma.

Introdução

- Grande parte dos vídeos disponíveis atualmente opera entre:

$$24 - 30\ FPS$$

- Monitores modernos frequentemente trabalham entre:

$$60Hz \rightarrow 144Hz$$

- Isso pode gerar:
  - baixa fluidez;
  - motion blur;
  - descontinuidade temporal.
- Video Frame Interpolation (VFI) busca sintetizar quadros intermediários artificialmente.

Problema Central
Modelos modernos de interpolação de quadros apresentam elevado custo computacional e consumo significativo de memória VRAM.

Desafio
Viabilizar a execução desses modelos em GPUs de consumo com aproximadamente 8 GB de VRAM.

Fonte: Elaborado pelo autor com base em Niklaus et al. (2017; 2021).

Hipótese Científica

Hipótese
A separabilidade matemática dos kernels, associada a um pipeline nativo em C++ e CUDA, possui potencial para reduzir a latência e o consumo de memória VRAM quando comparada a implementações baseadas em frameworks de alto nível.

$$O(N^2) \implies O(2N)$$

- menor custo computacional;
- menor footprint de memória;
- maior viabilidade em hardware de consumo.

Fundamentado em Niklaus et al. (2017).

Objetivos

Objetivo Geral
Avaliar experimentalmente os efeitos da convolução separável no desempenho computacional da inferência do modelo SepConv em GPU, utilizando implementação nativa em C++, com ênfase na latência e no consumo de memória VRAM.

Escopo
O foco do trabalho está exclusivamente na etapa de inferência, utilizando pesos pré-treinados disponibilizados na literatura.

Objetivos

1. Desenvolver pipeline em C++ e OpenCV;
2. Integrar ONNX Runtime para inferência do modelo SepConv;
3. Implementar kernels CUDA para convolução separável;
4. Comparar a solução com a baseline em Python e PyTorch;
5. Mensurar:
   - latência;
   - FPS;
   - consumo de VRAM.

Pesquisa Bibliográfica

Bases Consultadas

- Google Scholar;
- IEEE Xplore.

Operadores Booleanos

- AND
- OR

Strings de Busca

1. `("video frame interpolation"OR "frame interpolation") AND ("convolutional neural network"OR CNN OR convolution)`
2. `("video frame interpolation") AND ("adaptive convolution"OR "separable convolution")`
3. `("video frame interpolation") AND ("optical flow"OR convolution OR transformer)`

Fonte: Elaborado pelo autor.

Pesquisa Bibliográfica



Fonte: Elaborado pelo autor.

Trabalhos Relacionados

Autor	Estratégia	Limitação
Teed e Deng (2020)	RAFT baseado em fluxo óptico recorrente	Elevado custo computacional e alto consumo de VRAM
Niklaus et al. (2021)	Convoluções adaptativas separáveis	Dependência de frameworks de alto nível
Han et al. (2024)	Modelagem de Movimento Multi-Escala Pirâmide Hierárquica e Fluxo Intermediário	Melhor representação de movimentos complexos e redução de artefatos temporais.
Este Trabalho	**Inferência nativa em C++ e CUDA**	**Redução de latência e uso de VRAM**

Fontes: Teed e Deng (2020); Niklaus et al. (2021); Wijma et al. (2021).

Fundamentação Teórica

RAFT — Teed e Deng (2020)

- estima vetores de movimento;
- utiliza warping explícito;
- múltiplas iterações refinam o fluxo;
- elevado custo computacional;
- alto consumo de VRAM.

SepConv — Niklaus et al.

- não calcula fluxo explícito;
- movimento implícito no kernel;
- síntese e movimento integrados;
- menor footprint de memória;
- maior viabilidade computacional.

Diferença Central

O movimento deixa de ser representado por vetores explícitos e passa a ser modelado diretamente pelos pesos do kernel adaptativo.

Fonte: Teed e Deng (2020); Niklaus et al. (2021).

Convolução Tradicional

$$I'(x,y) = \sum I(x+i, y+j) \cdot K(i,j)$$

- kernel fixo;
- mesma filtragem para toda imagem;
- insuficiente para Video Frame Interpolation.

Fonte: Niklaus et al. (2017).

Convolução Adaptativa

O kernel passa a ser estimado individualmente para cada pixel.

- modela movimento;
- sintetiza o quadro intermediário;
- integra movimento e síntese.

Kernel Tradicional

$$K \in \mathbb{R}^{N \times N}$$

Para:

$$51 \times 51$$

temos:

$$2601$$

coeficientes por pixel.

Kernel Separável

A rede gera:

$$v \in \mathbb{R}^N$$

e

$$h \in \mathbb{R}^N$$

$$K = v \cdot h^T$$

Posto 1 (Rank-1)
O kernel bidimensional pode ser decomposto em dois filtros unidimensionais.

Fundamentação Teórica

Kernel Tradicional 3x3

$$\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Forma Separável

$$\begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 2 & 1 \end{bmatrix}$$

- convolução horizontal;
- convolução vertical;
- redução do número de multiplicações.

$$O(N^2) \implies O(2N)$$

Método	Operações	Complexidade
Kernel Tradicional	$51 \times 51 = 2601$	$O(N^2)$
Kernel Separável	$51 + 51 = 102$	$O(2N)$

- menos operações matemáticas;
- menor uso de memória;

Fundamentação Teórica

- Em GPUs modernas, o principal gargalo frequentemente não é a computação:

acesso à memória.

- Grandes tensores aumentam:
  - contenção de barramento;
  - tráfego PCI Express;
  - pressão sobre VRAM;
  - latência.

- Transferências excessivas entre RAM e VRAM degradam desempenho.

Relação com o Trabalho
A separabilidade reduz o footprint espacial e o tráfego de memória durante a inferência.

Fonte: Hennessy e Patterson (2011).

PSNR

- mede erro pixel a pixel;
- derivado do MSE;
- maior valor implica maior fidelidade visual.

SSIM

- avalia similaridade estrutural;
- baseado na percepção humana;
- detecta borramentos e artefatos.

Objetivo
Mensurar o equilíbrio entre qualidade visual e eficiência computacional.

Fonte: Wang et al. (2004); Gonzalez e Woods (2009).

Desenvolvimento

1. Leitura e decodificação dos quadros utilizando OpenCV;
2. Transferência dos dados para a VRAM;
3. Inferência do modelo SepConv utilizando ONNX Runtime;
4. Geração dos vetores separáveis horizontais e verticais;
5. Aplicação da convolução separável por kernels CUDA;
6. Síntese do quadro intermediário;
7. Transferência apenas do resultado final para o Host;
8. Codificação do vídeo resultante.

Objetivo
Minimizar transferências Host-Device e manter o processamento predominantemente na VRAM.

Metodologia Experimental

Dataset

- Vimeo-90K;
- cenas com movimento rápido;
- resolução Full HD.

Métricas

- PSNR;
- SSIM;
- FPS;
- uso de VRAM;
- latência.

Hardware

- GPU de consumo;
- aproximadamente 8 GB de VRAM.

Ferramentas

- NVML;
- cudaEventRecord;
- NVIDIA Nsight Systems.

Comparação
Comparação direta entre a implementação proposta e a baseline Python/PyTorch.

Cronograma

Atividade	M1	M2	M3	M4
Revisão Bibliográfica	X			
Pipeline OpenCV	X	X		
Integração ONNX Runtime		X	X	
Implementação CUDA			X	X
Coleta de Métricas			X	X
Redação Final				X

Referências

- Gonzalez, R. C.; Woods, R. E. Digital Image Processing. 2009.
- Hennessy, J.; Patterson, D. Computer Architecture: A Quantitative Approach. 2011.
- Niklaus, S.; Mai, L.; Liu, F. Video Frame Interpolation via Adaptive Separable Convolution. 2017.
- Niklaus, S. et al. Revisiting Adaptive Convolutions for Video Frame Interpolation. 2021.
- Teed, Z.; Deng, J. RAFT: Recurrent All-Pairs Field Transforms for Optical Flow. 2020.
- Wang, Z. et al. Image Quality Assessment: From Error Visibility to Structural Similarity. 2004.

Obrigado!

Dúvidas?